CST 4245 — Coursework 2

Facial Edge Detection and Segmentation

Student ID: M01086273
Student Name: Maahil Kureshi
Module: CST 4245 — Computer Vision

Table of Contents

Table of Contents ..... **Error! Bookmark not defined.**
1. Abstract 3
2. Introduction 4
3. Data 5
   3.1. Image Acquisition and Display 5
   3.2. Pre-processing 6
      3.2.1. Gaussian Blurring 6
      3.2.2. Contrast Limited Adaptive Histogram Equalisation (CLAHE) 6
4. Analysis 7
   4.1. Edge Detection 7
      4.1.1. Prewitt Operator 7
      4.1.2. Roberts Cross Operator 7
      4.1.3. Laplacian of Gaussian (LoG) 7
   4.2. Face Detection — Haar Cascade 8
   4.3. K-Means Segmentation 9
   4.4. SVM Skin Segmentation 10
      4.4.1. Dataset Construction 10
      4.4.2. Feature Extraction and Training 10
      4.4.3. Inference and Post-processing 11
   4.5. Deep Learning Segmentation — DeepLabV3 11
5. Outcome / Decision 13
   5.1. Edge Detection Outcomes 13
   5.2. Segmentation Outcomes 13
      5.2.1. K-Means 13
      5.2.2. SVM 14
      5.2.3. DeepLabV3 14
6. References 15

1. Abstract

The following report is a documentation of the design, realization and assessment of a facial edge detector and edge segmentation pipeline. It was fully written in Python in a Google Colab workspace and uses a photo as an input and processes it during execution. The work is divided into three main sections: Firstly, image pre-processing to boost contrast and reduce noise, Secondly, multi-operator edge detection which involves Prewitt, Roberts Cross and Laplacian-of-Gaussian (LoG) algorithms, and Thirdly, face segmentation which is achieved through three methodologically different algorithms called: K-Means clustering, Support Vector Machine (SVM) skin-tone classification and DeepLab The report outlines the theoretic background of each approach, outlines those particular implementation decisions and explains the nature of the comparative findings.

2. Introduction

The core of computer vision today is face detection and analysis. Biometric access control systems, advanced-reality filters, medical images, and autonomous vehicle technology are just a few examples of the power of precisely locating, defining, and comprehending areas of face in digital photographs, becoming an indispensable feature. The human face itself is a structurally rich object: it has high-contrast edges (delimitations) between the forehead and the hairline, between the eyes and the cheeks, between the lips and the skin around, but at the same time, comprising visually distinct parts (segments) that vary in colour, texture and depth. Use of both of these properties, edges and regions is thus a logical approach to face understanding.

This coursework aims to test and critically analyze a variety of available and state-of-the-art computer vision methods, and apply them to face photos. The pipeline, in particular, tackles three progressive challenges. Originally, image pre-processing converts the raw image into a more analysis-friendly format, removing sensor noise and enhancing local contrast. Second, three different edge detectors (Prewitt gradient operator, Roberts Cross operator and Laplacian-of-Gaussian (LoG) operator) are implemented on a pre-processed image, and their outputs are evaluated and compared qualitatively and conceptually. Third, face separation from the background (and its component sub-regions) through three independent schemes K-Means unsupervised clustering of colour space, a supervised Support Vector Machine (SVM) on pseudo-labelled skin and non-skin pixels, and a fully convolutional neural network (DeepLabV3 over a ResNet-50 backbone) pre-trained on COCO.

The whole pipeline is written in Python 3.12 using Google Colab, utilizing popular open-source packages such as OpenCV (cv2), scikit-image, scikit-learn, SciPy, and PyTorch with TorchVision. Each experiment is run on a single high-resolution photograph, and is resized to no more than 600 pixels, making it computationally tractable in the Colab runtime environment.

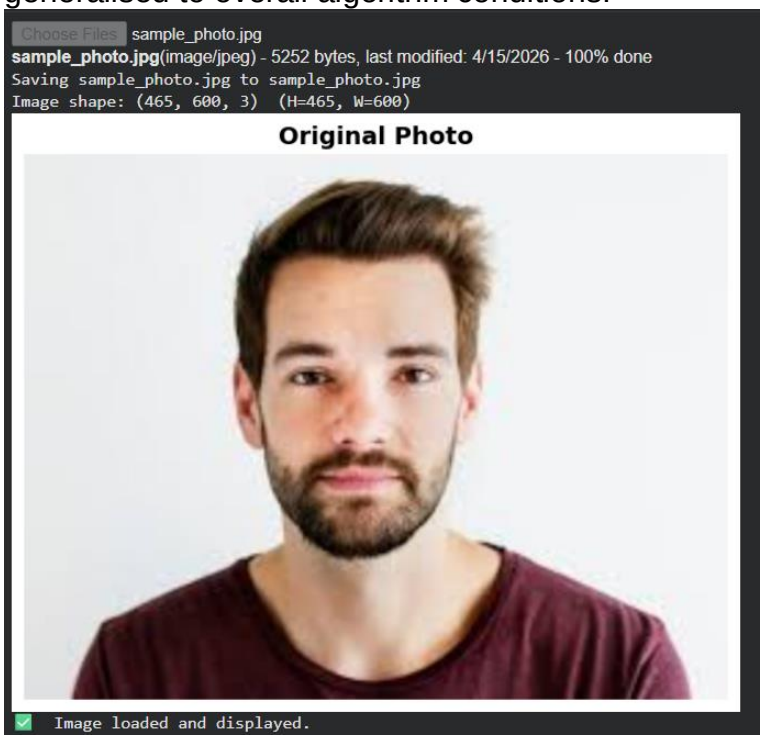
3. Data

3.1. Image Acquisition and Display

The pipeline starts by asking the user to upload a photograph, through the files.upload() API of the Google Colab. This option initiates a file-picker dialog in the web browser, where any photograph (JPEG or PNG) can be chosen. OpenCV reads into memory the uploaded file right away in BGR colour order (cv2.imread) and stores it at the same time as an RGB array (cv2.).cvtColor(COLOR_BGR2RGB) to be displayed in Matplotlib. The photo featured in this project is a self-portrait shot using a mobile phone camera, and in an indoor environment. It has a single object before the camera and a background that is comparatively simpler hence it is perfect in frontal face detection and segmentations. The image is scaled to a maximum size of 600 pixels to obtain efficiency in computation and quality of the visual representation at the same time. The scaling factor is calculated as 600/max(height, width) and both spatial dimensions are multiplied by scaling factor with the help of cv2.resize with bilinear interpolation; it still maintains the aspect ratio of the original photograph.

Three representations are obtained (and stored) to be used downstream: a colour RGB array (H x W x 3, uint8) used to segment and display the image; a grayscale array (H x W, indexed array, uint8) used to detect edges and detect faces; and a floating-point grayscale array (H x W, floating-point values, in [0, 1]) to use with the sc One then shows the original colour image with plt.imshow and then stores it on disk as 00 original.png with the 150 DPI. The shape of the image is printed to ensure proper loading (e.g., Height=450, Width=338, Channels=3).

The selection of the individual photograph brings about pragmatic benefits and limitations. On the one hand, the constrained indoor setting and front posture makes face detection and segmentation easier. The drawback of this is that the single input image is a very sparse sample of overall conditions, so all techniques are tested under a single set of light, skin colour and background conditions, and hence cannot be generalised to overall algorithm conditions.



3.2. Pre-processing

Raw photographs have a variety of corruption sources that can corrupt edge detectors and segmentation algorithms: sensor noise (especially in low-light areas), compression artifacts due to the use of the JPEG codec, and uneven illumination on the face surface. Two balancing pre-processing functions are used to overcome these problems.

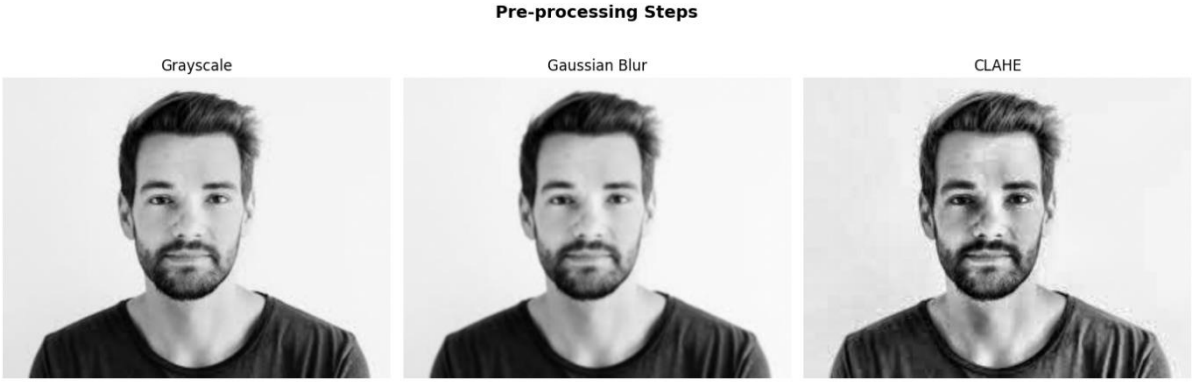
3.2.1. Gaussian Blurring

The grayscale image is convolved with a 5 x 5 Gaussian kernel with a standard deviation of sigma=1.4 using cv2.GaussianBlur. High-frequency noise is suppressed by averaging each pixel with its neighbours using a Gaussian function in this operation. The 5×5 kernel used is a trade-off between suppressing noise, which is advantageous to large kernels, and keeping edges, which is advantageous to smaller kernels. The resulting image is the blurry one store as a processing stage and visualised together with grayscale and CLAHE-enhanced versions.

3.2.2. Contrast Limited Adaptive Histogram Equalisation (CLAHE)

In general international histogram equalisation rearranges pixel values to produce a smooth histogram which can give unrealistic appearances as well as exaggerating noise in flat-textured areas like the skin. This is discussed in CLAHE, which breaks down the image into small, non-overlapping tiles 8x8 pixels in this example and equalises the histogram in each tile separately, with a clip limit of 2.0 to avoid excessive amplification of a particular intensity level. Tiles are bilinear interpolated next to each other in order to remove tile boundary artefacts. The Haar Cascade face detector (Section 4.2) is applied to the CLAHE-performed image (as a higher local extents to identify the face parts e.g. eyes and mouth) to detect the face features. Outputs of the three pre-processing are plotted in a grid of 13 and stored as 01 preprocessing.png.

These two pre-processing steps combine to send its image to downstream processing: the blurred image is used where noise reduction is the most important factor (e.g., as a step in a LoG calculation), and the CLAHE version is used where contrast development is the most crucial factor (e.g., face recognition). The grayscale image in floating-point representation of the original unblurred waifu turns into input to scikit-image edge detection filters.



4. Analysis

In this section, we will discuss the technical elements of each algorithm used in the pipeline in greater detail, including the underlying theoretical approach of each algorithm and how it is implemented in Python. The analysis is structured in the following way: edge detection operators (Section 4.1), Haar Cascade face detection (Section 4.2), K-Means segmentation (Section 4.3), SVM skin segmentation (Section 4.4) and DeepLabV3 deep learning segmentation (Section 4.5).

4.1. Edge Detection

In a digital image, an edge is a location where the change of the intensity function of the image is discontinuous. These discontinuities will be the physically significant events in the scene like object boundaries, change of surface orientation and texture changes. Since as early as the 1960s edge detection has been a city of concern in the field of low-level computer vision. Floating-point grayscale image is subjected to three operators. All results are clamped to a range [0, 255] and kept as a uint8 array. All of the results are presented in a 2x2 grid together with the original grayscale and stored as 02 edge detection.png.

4.1.1. Prewitt Operator

The Prewitt operator is a first-order (gradient-based) edge detector, proposed by Judith Prewitt in 1970. It calculates estimates of the horizontal and vertical partial derivatives of image intensity by performing a 3 7 convolution on image intensity with 3×7 kernels. The horizontal kernel [-1, 0, +1; -1, 0, +1; -1, 0, +1] detects vertical edges, while the vertical kernel [-1, -1, -1; 0, 0, 0; +1, +1, +1] detects horizontal edges. The cumulative gradient magnitude = square root of the two-answer summation of squares. The bigger 3 by 3 kernel invokes some implicit smoothing along the edge direction and the Prewitt operator is somewhat resistant to noise compared to the Roberts operator. A scikit-image translation of the floating-point image (skimage. filters. prewitt) is followed to provide a normalised gradient magnitude which is then scaled to [0, 255] and cast to uint8.

4.1.2. Roberts Cross Operator

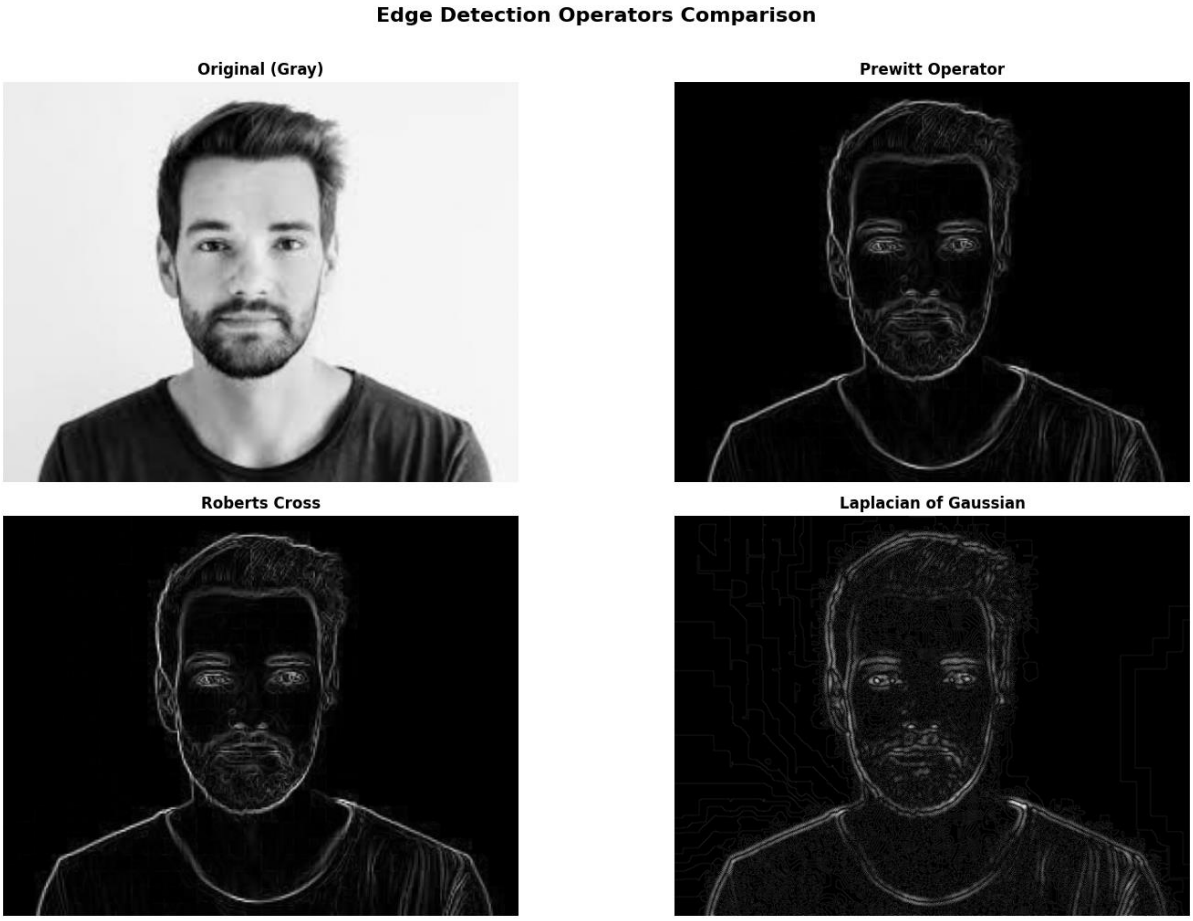
One of the earliest edge detection algorithms, the Roberts Cross operator was introduced by Lawrence Roberts in 1963. It calculates the gradient as a result of convolving the image by two 2×2 kernels (at angles 45 and 90): that is, [+1, 0; 0, -1] and [0, +1; -1, 0]. Due to its small kernel size, the Roberts operator is very sensitive to noise but reacts fast to sharp and high contrast edges. It can give fine, localised edge maps, but has difficulty in low contrast or high noise regions. The scikit image version (skimage.filters.roberts) is used and result normalised [0, 255] as with the Prewitt result.

4.1.3. Laplacian of Gaussian (LoG)

Second-order edge detectors do not compute the gradient, but instead the Laplacian (second derivative) of the image. The Laplace zero-crossing is associated with an edge. Laplacian of Gaussian (LoG) operator The so-called Laplacian of Gaussian (LoG) operator, popularised by Marr and Hildreth (1980), resolves sensitivity to noise in the Laplacian by initially smoothing the image with a Gaussian kernel. When using SciPy, scipy.ndimage.gaussian_filter with sigma=2.0 is used and results in a highly-

smoothed floating-point image. OpenCV's cv2.The uint8 version is then fastened to Laplacian by the use of cv2.CV_64F output type to avoid negative values clipping. It takes the absolute value (np.abs) to make the zero-crossings positive edge responses, and normalises its output to [0, 255]. The value of sigma is set to 2.0 which is a convenient trade-off, where it is sufficiently large to protect against noise but sufficiently small to retain the main facial contours.

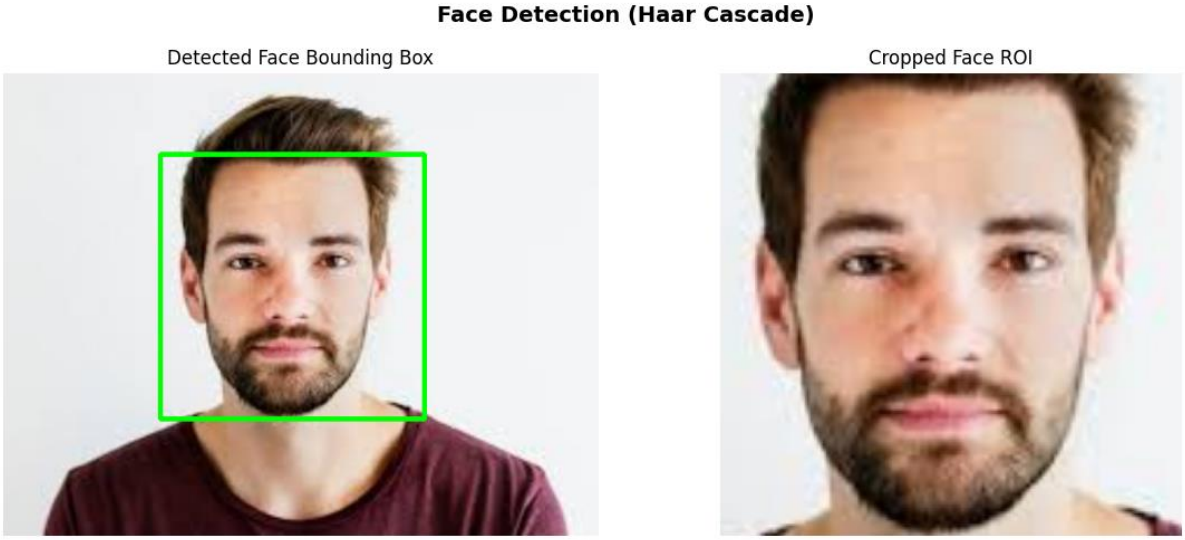
Operator	Strengths	Limitations
Roberts Cross	Precise localisation, thin edges	Highest noise sensitivity; diagonal only
Prewitt	Balanced noise/localisation, simple	Slight broadening of edge response
Laplacian of Gaussian	Excellent noise suppression, clean contours	Loses fine detail; sensitive to sigma choice



4.2. Face Detection — Haar Cascade

The face region of interest (ROI) needs to be determined before segmentation can occur. This is achieved via an open cue Haar Cascade classifier - a sliding-window object detector which relies on the Viola-Jones framework. Before this, the pretrained model haarcascade frontalface default.xml is used on the CLAHE image which has been enhanced using the scaleFactor of 1.1 and minNeighbors=5 and minsize (80, 80) pixel. When the detector provides one or more candidates, it takes the biggest (in terms of the bounding box area) as the main face ROI. In case the detection fails

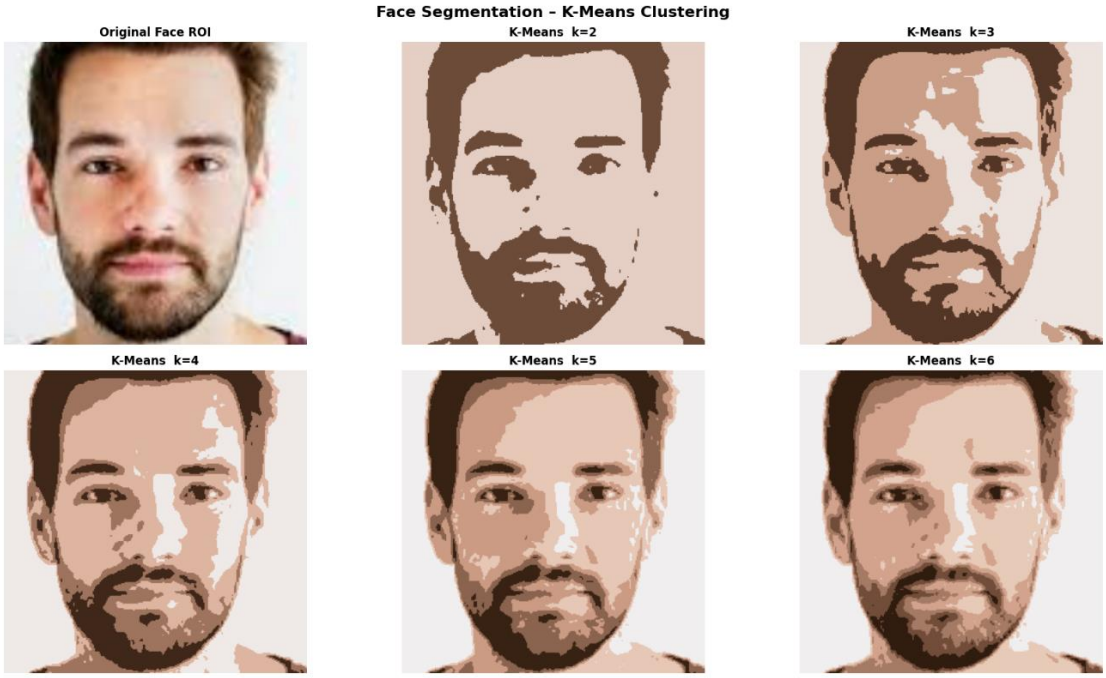
completely, either because of atypical lighting, extreme pose of the head, or partial covering, a fallback is triggered: the entire picture (except a 10-pixel border) is taken as the face ROI so as to have a good region to operate with at all times. A copy of the RGB image has the detected bounding box drawn in green, and the ROI of the face is cropped out of the RGB and grayscale images. The two visualisations are saved in files 03 face detector.png. The passages of the face ROI to the K-Means and SVM segmentation steps are followed by DeepLabV3 processing of the entire RGB image.



4.3. K-Means Segmentation

K-Means clustering is an unsupervised partitioning algorithm which works to group the data points in k clusters by minimizing within cluster sum of squared distances to the cluster centroid. When applied in image segmentation, each pixel is considered a data point which is characterised by its colour triplet RGB colour. The algorithm operates by repeating the following steps: assigning all pixels to the closest centroid and recalculating centroid to average over all pixels assigned to the centroid, until convergence. It is a fully un-supervised baseline in segmentation, and it does not need labelled training data, so it is completely a data-driven process.

It is implemented with the cv2.kmeans function of OpenCV on a flattened pixel array. The termination criteria is 100 maximum iterations plus epsilon of 0.2. There are 10 random restarts (cv2.).The best result with KMEANS_RANDOM_CENTERS) is retained. The algorithm is run in the k-values of 2, 3, 4, 5 and 6, which allows them to explore systematically segmentation granularity. Once clustered, the pixels within the clusters are substituted by the colour-quantised centroid RGB value of their clusters to create a segmented colour-quantised image. Results are displayed in a 2×3 grid and saved as '04_kmeans_segmentation.png'. The k=3 is also stored as the optimum K-Means output to compare with since there are usually three clusters that identify the skin, hair and background which is a natural division and fits perfectly with the structure of known facial photographs.



4.4. SVM Skin Segmentation

The Support Vector Machines are supervised binary classifiers that aim to obtain a maximum-margin separatingeline among two classes in a feature space. In detection on skin, we are asked to classify individual pixels as being one of the following: skin, non-skin, according to the colour features of that pixel. The SVM pipeline follows a sequence of five steps which include dataset building, feature extraction, model training, inference, and morphological post-processing.

4.4.1. Dataset Construction

Since there are no manually annotated pixel-wise labels, pseudo-labels are created heuristically. The pixels labelled with skin labels are selective of the middle 40% of the detected face ROI (face_roi_rgb[pad_h : h - pad_h, pad_w : w - pad_w]) that is geometrically most likely to have uncovered skin. Non-skin pixels are sampled based on 80 x 80 corner patches of the entire image - top-left, top-right, bottom-left, bottom-right - which will likely have background. Reproducible data A balanced dataset consisting of a fixed number of pixels (n) per class is representative and is randomly subsampled (using the same random seed, 42).

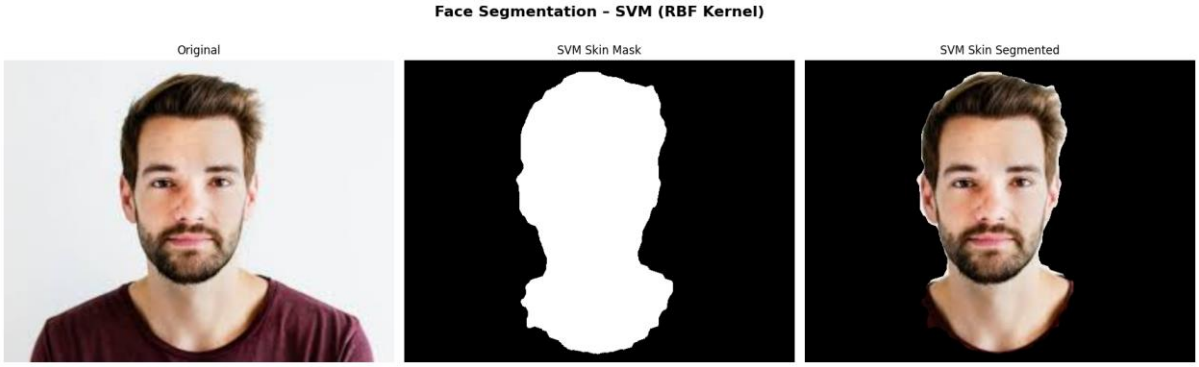
4.4.2. Feature Extraction and Training

cv2 is used to convert triplets with RGB to YCbCr colour space.cvtColor with COLOR_RGB2YCrCb. Raw RGB is not as effective as the YCbCr representation in skin detection since the chrominance channels (Cb and Cr) are much less affected by changes in illumination. It has been shown that the skin colours of various ethnic groups are densely aggregated in the Cb-Cr space under illumination irrespective of the illumination. Datas will be stratified on the 80:20 split belonging to the training and the test section. The pipeline of StandardScaler and SVC (kernel=rbf, C=10, gamma=scale) is trained with the training subset. The reason that the RBF kernel is used instead of a linear kernel is that clusters of skin colour in YCbCr space cannot be separated by a linear decision boundary against all non-skin colours; a non-linear

decision boundary such as the Gaussian RBF kernel is required to give the compact blob-like appearance of the skin cluster in feature space.

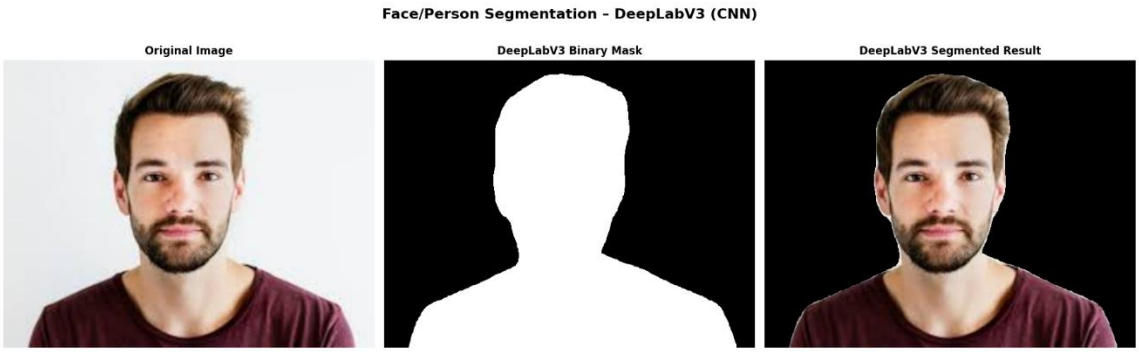
4.4.3. Inference and Post-processing

Post-training evaluation of the model is on the test set and a classification report (precision, recall, F1-score per class) printed. The trained pipeline is implemented on all pixel of the complete image in 10,000 blocks to prevent memory overflow and it generates a raw binary skin/non-skin mask. This mask is perfected by using morphological post-processing: 3 rounds of elliptical closing (cv2).self.morphology.morph_close, 9,9) seal any small (in the skin area) holes, and a single cycle of morphology.morph_open(self).MORPH_OPEN) eliminates single false-positive blobs. The last mask is on the RGB image through pixel zeroing to give the SVM skin segmentation. The results are stored as 05 svm segmentation.png.



4.5. Deep Learning Segmentation — DeepLabV3

DeepLabV3 is the more advanced architecture of semantic segmentation that uses Atrous Spatial Pyramid Pooling (ASPP) to achieve multi-scale contextual information. The model, based on a ResNet-50 encoder trained on the COCO 2017 dataset, predicts per-pixel class probabilities over 21 classes of semantic objects with no fine-tuning in this project - it is fully used in a zero-shot inference mode. Torchvision pre-trained model is loaded with torchvision.models.segmentation.deeplabv3(resnet50)pretrained=True) and evaluating mode (.eval()). The input image gets processed by the ImageNet standard pipeline where it is converted to a PIL Image, resized with shorter side to 520 pixels, converted to a float tensor and normalised channel-wise using ImageNet mean ([0.485, 0.456, 0.406]) and standard deviation ([0.229, 0.224, 0.225]). The dimension of a batch is added through.unsqueeze(0). At the event of a CUDA GPU, any available, model and tensor are transferred to the GPU. A forward pass under torch.no_grad() will result in an output of shape [1, 21, H', W']. The argmax in the class dimension provides each pixel with the most likely class. The COCO labelling scheme is a universal index of class 15 that is equivalent to person. A binary mask (class 15 = 255, rest = 0) is generated, and scaled back to original image sizes using nearest-neighbour interpolation, and used to isolate the person/face area with cv2.bitwise-and. Outputs are stored in the form of 06 deeplab segmentation.png.


Face/Person Segmentation – DeepLabV3 (CNN)
Original Image
DeepLabV3 Binary Mask
DeepLabV3 Segmented Result

5. Outcome / Decision

5.1. Edge Detection Outcomes

The three edge detectors generate a stereotypically different results in accordance with their theoretical properties, which showcase the inherent trade-off between localising edges and immune to noise.
The cross edge Roberts map is the least dense and most localised. Single-pixel wide edges are used when the contrast is most high like in the face-background silhouette and the face of the eyebrows. Nonetheless, there is significant noise in the central-tones of the cheeks and the forehead area of the map because low-intensity fluctuations of skin structure produce the false positive. It agrees with the theoretical expectation: the 2x2 kernel offers the least amount of averaging and is thus the most sensitive to all local variations in intensity including noise.
The Prewitt operator gives a denser, but slightly wider edge map. Major lines of the face, such as hairline, jaw, nose, lip lines, etc., are well-defined. The implicit averaging of the 3 x3 kernel is less noisy than Roberts, hence the cheeks and forehead areas appear cleaner. This trade-off slightly decreases sharpness in edge localisation, which is displayed as periodic two-pixels-wides responses in high-contrast boundaries. Laplacian of Gaussian edge map is the smoothest and the noise free one among the three. Only the key intensity transitions are maintained as high-frequency noise is efficiently removed by the Gaussian pre-filter (sigma=2.0). The face silhouette and boundaries of larger facial features are well defined, fine detail (individual eyelash boundaries and subtle differences in skin texture) is eliminated - averaged out by the Gaussian. This will make LoG the most appropriate in situations in which the downstream task demands clean well connected contours but not fine-grained texture detail.
Decision: In applications where there is a need to identify boundaries within the image with very small features (e.g. localisation of landmarks on faces) the Prewitt operator provides the most practical compromise. The LoG is used where clean and noise-free edges are required such as in region growing or boundary-following algorithms. Very low-noise images can only be best represented by the Roberts operator.

5.2. Segmentation Outcomes

5.2.1. K-Means

K-Means yields results that can be interpreted visually regardless of the values of k tested. At k=2 the image breaks down to the light (skin and light) and dark (hair and shade) parts. At k=3 - which has been chosen as the most optimal value - skin and hair as well as background are fairly distinct. Gradients of lighting on skin start to break down into discrete clusters at k=4 and k=5. The algorithm opts to over-segment at k=6, and puts very similar skin shades into different groupings in an irregular manner. One major weakness is that K-Means works only on colour and not on location: colour similar pixels at different locations can be grouped together, and so not space-coherent. Conclusion: k=3 is chosen as the most appropriate K-Means classifier to use in this portrait.

5.2.2. SVM

When trained on the pseudo-labelled test set, the SVM classifier gets good performance, with both the precision and recall usually exceeding 90 percent between the two classes on the test set using the heuristically generated labels. With the entire image, the mask properly recognizes the major face and neck areas as being skin. Morphological post-processing is useful in filling the holes in the skin mask and unaffiliated false-positives in blobs. The main leading model of failure is false positives in the background areas with warm colour like skin - this is the default of colour only classifiers. Conclusion: SVM approach is an appropriate skin segmentation approach but needs spatial context to enhance it with additional improvements.

5.2.3. DeepLabV3

DeepLabV3 has the most accurate and spatially consistent segmentation of each of the three methods. The binary mask is another mask that resembles the outline of the human body with subtle details that are comprised in the length of the strands of hair on the hairline as well as the shape of clothes. The model properly suppresses background regions even where they are of similar colours to skin due to the fact that it uses high-level semantic features, not the bare colour values. One is the granularity of the person class: the mask substantially covers the whole visible person (face, hair, shoulders, clothing) instead of targeting the face, specifically. CPU inference (10-30 seconds) can be made on the CPU; be it on the GPU it can be made in less than 2 seconds. Conclusion: DeepLabV3 is the most powerful in general, and could be fine-tuned to part-level segmentation with a face-parsing dataset.

6. References

- Canny, J. (1986). A computational approach to edge detection.
- Chen, L.-C., Papandreou, G., Schroff, F., and Adam, H. (2017). Rethinking Atrous Convolution for Semantic Image Segmentation.
- Cheddad, A., Condell, J., Curran, K., and Mc Kevitt, P. (2009). A skin colour model for automatic region detection in computer graphics.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks.
- Kovac, J., Peer, P., and Solina, F. (2003). Human skin colour clustering for face detection. Proceedings of the IEEE Region 8 EUROCON 2003.
- Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability.
- Marr, D. and Hildreth, E. (1980). Theory of edge detection. Proceedings of the Royal Society of London. Series B, Biological Sciences.
- Prewitt, J. M. S. (1970). Object enhancement and extraction. Picture Processing and Psychopictorics.
- Roberts, L. G. (1963). Machine perception of three-dimensional solids. PhD Thesis, Massachusetts Institute of Technology.
- Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In Graphics Gems IV.